

Smart Helmet with IoT: project report

Problem, features, intelligence system, parts, architecture and build path. Prepared 2 October 2026.

1. Overview

Motorcycle riders face several risks at once: riding without the helmet properly on or after drinking, drowsiness, vehicles in the blind spot, bad roads, and a crash that nobody learns about quickly, with no record and no sense of how serious it was.

The smart helmet carries an ESP32-based controller that senses the rider and the road. It warns the rider, blocks starting when needed, and after a crash judges the severity and sends an alert with evidence.

Core idea: deterministic rules make every safety decision. Small ML models help only where patterns are fuzzy (nods, wet roads), and heavy ML runs offline on logged data.

Scope used in this report: Tier 1 has eight features (worn check, alcohol check, crash detection with severity, GPS, LTE alert, black box, sleep detection, blind spot). Two cameras and road-condition analysis are additions built after Tier 1 works. The source summary still lists sleep detection and blind spot in Tier 2; this report follows your later decision to move them to Tier 1.

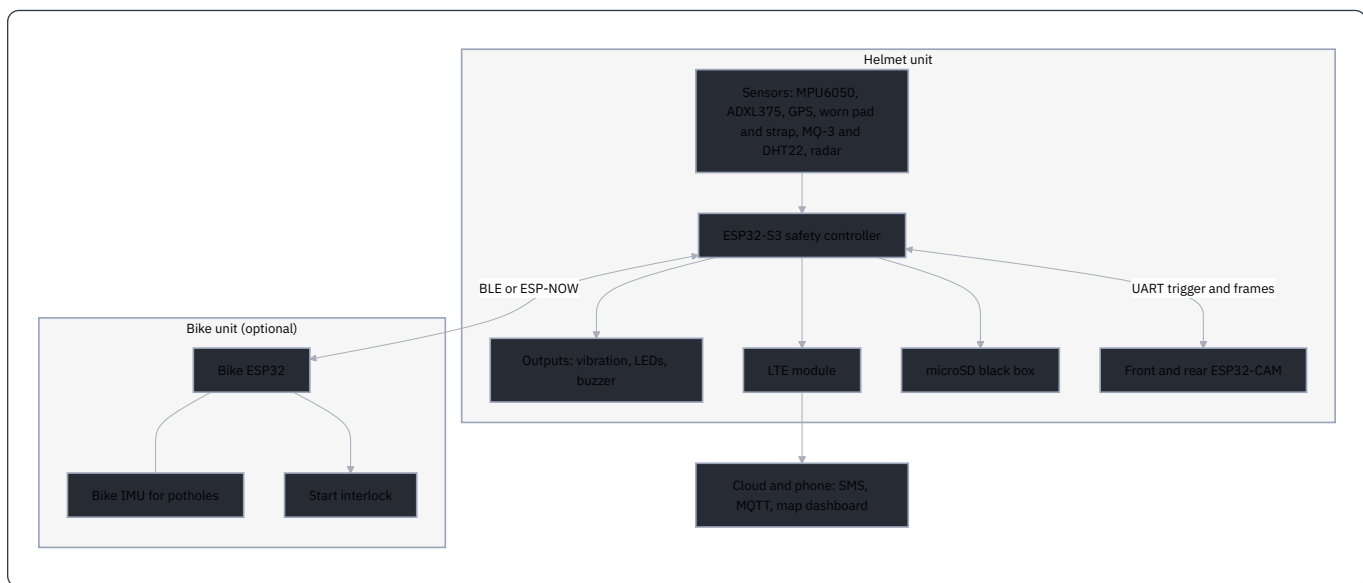
2. Problems and the features that solve them

Problem	Feature	Sensors	How it works	Main limit
Helmet not worn properly	Worn check	Capacitive pad, chin-strap switch	Two signals must agree, with debounce and a baseline set at power-on. The bike relays an "allow start" message over BLE or ESP-NOW.	Sweat, sunlight a hand can fool a single sensor
Drinking and riding	Alcohol check (start only)	MQ-3, DHT22, ADS1115	Warm-up lockout, clean-air baseline, Rs/RO ratio, temperature and humidity compensation, breath-peak	Screening, not measurement. Humid helmet a breathing techn

			threshold. Blocks starting only, never the engine while moving.	change readings
Crash goes unreported	Crash detection	ADXL375, MPU6050, GPS	Rule-based state machine: speed gate, high-g impact trigger, filtered tilt, stillness check, then a cancel countdown.	Thresholds must come from your drop and pendulum tests
Nobody knows how bad it is	Severity score	ADXL375, MPU6050, GPS	HIC15 plus pre-impact speed, final orientation, stillness and rider response, combined into minor, moderate or severe.	A triage hint, not a medical assessment
Help is not called	Emergency alert	LTE module (A7670C)	SMS first (location link, severity, speed), then HTTPS or MQTT data and JPEG frames, with a retry queue.	Needs coverage and a stable power supply
No record of what happened	Black box and two cameras	microSD, PSRAM buffer, front and rear ESP32-CAM	Ring buffer of IMU, GPS and state data, frames captured on the crash trigger, hash-chained log entries, then cloud sync.	Helmet may face the ground. Camera power can brown out
Drowsy riding	Sleep detection	MPU6050	Head-pitch estimate by sensor fusion, then nod detection (pitch drop for a set time, repeated in a window), calibrated per rider. Vibration plus tone.	Accuracy untested on real drowsiness
Vehicles in the blind spot	Blind spot assist	HLK-LD2451 radar, MPU6050	Radar gives distance, speed and direction. The ESP32 keeps approaching targets only, applies a distance gate and persistence, and ignores readings when head angle or lean is out of limits. Output: directional vibration and LEDs.	Head turns, lean and the $\pm 15^\circ$ beam
Potholes	Pothole logging (addition)	Bike IMU, GPS, camera optional	High-pass filtered vertical acceleration, speed-normalised peaks, speed-bump rejection,	Needs a bike-mounted IMU, so the head softens

			GPS tagging, repeated hits aggregated on a cloud map. Frames confirmed offline.	shocks
Wet roads	Wet-risk score (addition)	Rain sensor, DHT22, weather data, camera	Weighted low, medium or high score with time-since-rain decay, plus a wet/dry classifier on frames taken while the head is level and forward.	Advisory only. G and night hurt th camera

3. System architecture



Roles of the units

- **Helmet ESP32-S3:** the safety controller. Runs sampling, the crash state machine, alerts and communications as separate tasks.
- **ESP32-CAM boards:** separate boards so a camera brownout cannot reset the crash logic. Stills only, each on its own stable 5 V supply.
- **Bike ESP32 (optional):** start interlock and the pothole IMU. It only blocks starting.
- **Cloud:** MQTT or HTTPS ingest, database, map dashboard, pothole heatmap, ride history.

Crash response flow

1. Speed gate, worn check and impact trigger fire.
2. Tilt and stillness are confirmed.
3. A buzzer and vibration countdown starts, and the rider can cancel.
4. If not cancelled, severity is assessed and the SMS goes out.

4. If not cancelled, severity is scored and the SMS goes out.
5. Front and rear frames and the black-box data are saved and uploaded.
6. The system retries until delivery succeeds.

Links and security: LTE (SMS, HTTPS, MQTT), BLE or ESP-NOW helmet to bike, UART between boards. TLS, authenticated pairing and hash-chained logs.

4. Parts and their roles

Part	Role	Feature
Controllers		
ESP32-S3 dev board	Main brain: sensors, state machines, TinyML, communications. PSRAM helps image and data buffers.	All
ESP32-CAM with OV2640 (2)	Front and rear stills after a crash	Black box
Bike ESP32 (optional)	Start interlock, bike IMU, relay	Alcohol, potholes
Sensors		
ADXL375 (± 200 g)	Impact spike, HIC15 input	Crash, severity
MPU6050	Head tilt, nod, head direction, post-crash stillness, rotation	Crash, sleep, blind spot
NEO-6M GPS	Location, speed gate, speed lost in the crash	Crash, severity, alert
Capacitive pad and chin-strap switch	Detect a head inside and a fastened strap	Worn check
MQ-3, DHT22, ADS1115	Alcohol sensing, compensation and stable ADC readings	Alcohol check
HLK-LD2451 radar	Range, speed and direction of vehicles behind	Blind spot
Rain sensor, bike MPU6050	Rain on helmet, pothole shocks (not priced yet)	Road condition
Communication and storage		

A7670C LTE module and SIM	SMS and data alert after a crash	Alert
microSD module and card	Local black box log	Black box
Alerts and inputs		
Vibration motors, LEDs, buzzer	Sleep and blind spot warnings, warm-up and countdown sounds	Sleep, blind spot, crash
Cancel button	Stops a false crash alert and counts as rider response	Crash, severe
Power		
18650 cell, TP4056, 5 V boost	Battery, safe charging, 5 V for radar and MQ-3 heater	All
Separate LTE supply and large capacitor	LTE peaks near 2 A and cameras draw up to 500 mA each, so they need their own headroom	Alert, camera

Prices for the Sri Lankan parts are in the price list page: <https://claude.ai/artifact/DqUrfgHVgTnmWjPUJDwH7n>. That list has a 30-pin ESP32 (Rs 1,450), not the ESP32-S3 recommended here, so update it before ordering. The BME280 is a more precise upgrade to the DHT22.

5. Intelligence system

Build the rule-based baseline first and add ML only after you have real data to train on.

Four layers

Layer	Runs on	Purpose	Examples
1. Reflex (rules)	ESP32, offline	Deterministic safety decisions	Crash state machine, severity, worn ch alcohol, blind spot
2. Edge ML (TinyML)	ESP32-S3	Fuzzy pattern recognition	Head-nod classifier, wet/dry road classifier
3. Offline ML	Laptop or cloud	Heavy models and aggregation	Pothole detection (YOLO), pothole clustering, wet-risk model

4. Learning loop	Laptop or cloud	Improve over time	Collect, label, retrain, update firmware
------------------	-----------------	-------------------	--

Intelligence per feature

Feature	Type	Approach
Crash detection	Rules first	Threshold state machine. A classifier can come later, trained on staged tests (drops, pendulum, bumps).
Severity	Rules, no ML	HIC15 plus speed, orientation, stillness. No injury labels exist, so calibrate with drop tests.
Drowsiness	Rules, then TinyML	Window the pitch data (nod count, dwell time, variance). Version trains a small classifier on simulated nods and normal riding.
Blind spot	Rules	Radar filtering plus IMU gating. ML would be over-engineering.
Potholes	Rules, then offline ML	IMU events by rules, then offline YOLO on bump-triggered frame. Geo-tagged events clustered on the cloud map.
Wet road	TinyML plus score	Image classifier on the road region, fused with rain, humidity and weather data.
Alcohol	Calibration, no ML	Baseline and compensation curve.

Severity scoring

Peak acceleration alone is a flawed measure. HIC15 combines peak and duration over a window of at most 15 ms, and the source summary cites reference values of about 300 (minor concussion), 600 (moderate injury) and 1000 (life-threatening). Its main limit is that it uses linear acceleration only and ignores rotation. I have not verified those figures independently.

Class	Typical triggers
Minor	Low HIC15, rider responds or moves after the countdown
Moderate	Mid HIC15, higher pre-impact speed, or rider down but moving
Severe	High HIC15, or high speed followed by no movement, or rider horizontal and still

Thresholds are starting points to calibrate with your drop tests. A shell-mounted sensor is not a headform sensor, and the liner reduces what it sees (assessment). The alert should say "severe impact detected," not "rider is injured."

How to build the learning part

1. Build the baseline and log raw IMU, GPS and events to the SD card.
2. Collect labelled rides, including negatives such as helmet drops, speed bumps and hard braking.
3. Extract features and train, for example with Edge Impulse, using data from several riders.
4. Deploy as an Arduino or C++ library.
5. Validate with a confusion matrix and false alarms per hour on riders and routes the model never saw.
6. Run the model in shadow mode beside the rules before it can trigger anything.

Public data and memory limits

- **Wet/dry:** the RSCD and RoadSaW datasets come from vehicle cameras, so expect a gap with helmet footage. Pretrain, then fine-tune on your own frames.
- **Potholes:** public sets exist (a Roboflow pothole set and an IEEE DataPort set). Train YOLOv8 on them, then fine-tune on your footage.
- **Memory:** a small image classifier can need several hundred KB of RAM and ROM, and the original ESP32-CAM struggles. Use an ESP32-S3 with PSRAM, or process frames offline.

Safety rules for ML: it is never the sole trigger for a crash alert, and the speed, worn, impact and stillness gates must still pass. Every output needs a confidence threshold and smoothing. Fall back to the rules when the model is uncertain, and log every decision. Simulated nods differ from real drowsiness, so state that limit in the report.

Tools: Edge Impulse or TensorFlow Lite Micro; Python (scikit-learn, pandas), Ultralytics YOLOv8 and Roboflow offline; MQTT, a database, a map dashboard and DBSCAN-style clustering for pothole hotspots.

6. Build path

Phase	Goal	Done when
0	Power budget and ESP32 base	Stable runtime. no brownouts

1	MPU6050, ADXL375 and GPS logging to microSD	Raw data from drop and pendulum tests
2	Crash logic, severity, cancel button, LTE SMS	Simulated fall sends SMS with location and level
3	Worn check and alcohol lockout	Warm-up lockout and start-only block work
4	Sleep detection	No warning works for several riders
5	Black box and the bike ESP32 link	Data survives a drop
6	Blind spot radar	Road-tested alerts with head gating